Zephyr RTOS Student & Beginner's Guide

Hands-On Embedded Systems Engineering with Microchip PIC32CM GC00 Curiosity Pro

Board: pic32cm_gc00_cpro	**MCU:** PIC32CM5112GC00100	**Architecture:** ARM Cortex-M23 @ 48 MHz	**Document Version:** 1.0

CURRICULUM OBJECTIVE

This comprehensive laboratory guide introduces university students, electrical engineering candidates, and embedded developers to modern, production-grade real-time systems using **Zephyr RTOS** on the **Microchip PIC32CM GC00 Curiosity Pro** evaluation board.

1. Introduction to Real-Time Operating Systems (RTOS)

1.1 What is an RTOS?

An **RTOS (Real-Time Operating System)** is an operating system engineered to execute tasks with strict, deterministic timing guarantees. While general-purpose operating systems (Windows, macOS, Linux) optimize for maximum data throughput and user multitasking, an RTOS guarantees that hardware events, sensor readings, and interrupts are serviced within exact microsecond or millisecond boundaries.

1.2 Bare-Metal vs. RTOS Paradigm

Traditional embedded education begins with **Bare-Metal** programming, where firmware executes inside a single `while(1)` super-loop:

- **CPU Wastage:** Delays are executed using blocking loops (e.g. `for(int i=0; i<100000; i++);`), running the core at 100% load and wasting battery energy.
- **Inflexible Concurrency:** Adding simultaneous tasks (e.g. blinking an LED while sampling an ADC and receiving UART packets) requires complex interrupt state machines.
- **Zero Portability:** Firmware code directly writes vendor-specific registers (e.g., `PORTA->DIRSET.reg = (1 << 20);`). Migrating to a new microcontroller requires rewriting everything from scratch.

In contrast, an **RTOS** provides a **preemptive real-time scheduler**. Each task runs in its own thread with a private stack. When a thread pauses (e.g. `k_msleep(350)`), the kernel yields the CPU to lower-priority threads or places the MCU into an ultra-low-power idle state until the hardware timer interrupt wakes it.

1.3 What is Zephyr RTOS?

Zephyr is an open-source, scalable, highly secure RTOS hosted by the **Linux Foundation** and governed by industry leaders including Microchip, Google, Intel, Nordic Semiconductor, NXP, STMicroelectronics, and Qualcomm. Licensed under the permissive **Apache 2.0** license, Zephyr brings the proven software engineering architecture of the Linux Kernel (Devicetree, Kconfig, modular drivers, POSIX compatibility) down to 32-bit microcontrollers.

2. Why Choose Zephyr? (Purpose & Industry Advantages)

Feature	Bare-Metal (Traditional)	FreeRTOS	Zephyr RTOS
Hardware Abstraction	Direct register manipulation	Vendor HAL dependent	**Devicetree (DTS) Standard**
Build & Configuration	Manual C header `#define`s	`FreeRTOSConfig.h`	**Granular Kconfig Engine**
Cross-Vendor Portability	Zero (Locked to 1 chip)	Low (Requires vendor glue)	**Universal standardized APIs**
Power Management	Manual sleep register writes	Tickless idle (requires glue)	**Native Power Management Subsystem**
Protocol Stacks	Requires 3rd-party integration	External ports required	**Built-in BLE, Wi-Fi, CAN, TCP/IP, USB**

3. Hardware Architecture: PIC32CM GC00 Curiosity Pro

The **PIC32CM GC00 Curiosity Pro** evaluation board features the Microchip **PIC32CM5112GC00100**, a 32-bit ARM Cortex-M23 microcontroller equipped with advanced analog and security peripherals.

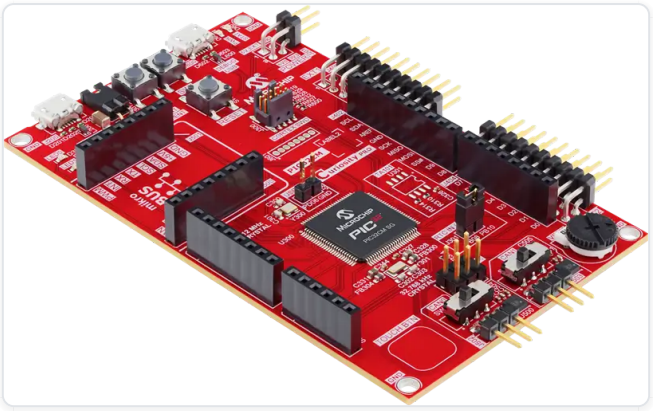


Figure 1: Microchip PIC32CM GC00 Curiosity Pro Evaluation Kit with Integrated nEDBG Debugger.

Specification	Hardware Value	Educational Significance
Processor Core	ARM Cortex-M23 (ARMv8-M)	Modern low-power 32-bit RISC architecture with security extensions
Clock Frequency	48.0 MHz (FDPLL96M)	High-speed deterministic instruction execution
On-Chip Flash	512 KB	Stores the Zephyr kernel, RTOS stacks, and user application firmware
SRAM	128 KB	Zero-wait-state memory for thread stacks, heaps, and variables
Onboard Debugger	nEDBG (CMSIS-DAP)	Enables flashing and hardware debugging without external programmer tools
Primary User LED	**Yellow LED0 on PA20**	Wired in **Active-Low** electrical configuration

3.1 Demystifying Active-Low Electrical Logic

A critical electrical engineering concept is understanding why the onboard LED on the Curiosity Pro is wired as **Active-Low** rather than Active-High:

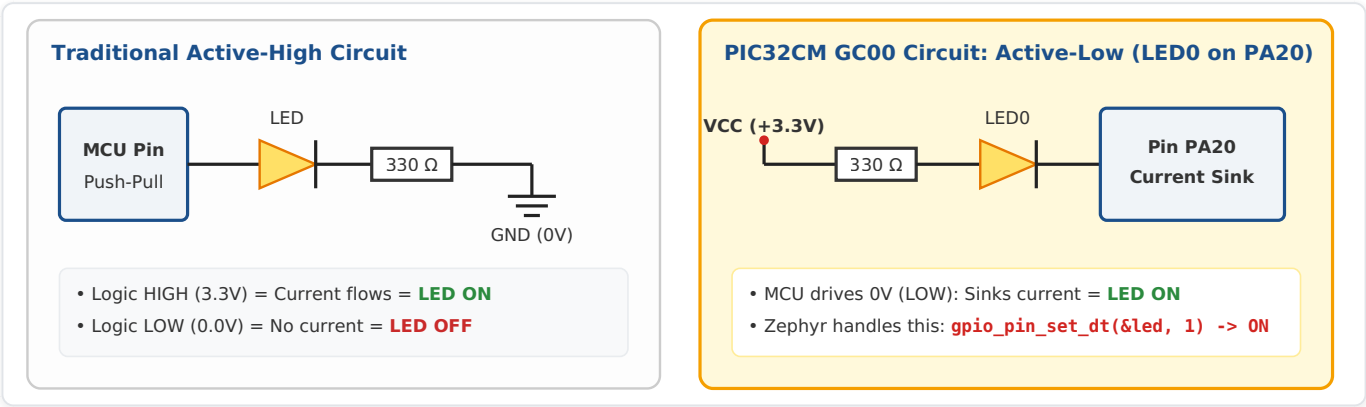


Figure 2: Electrical schematic comparison between traditional Active-High vs. PIC32CM GC00 Active-Low circuitry.

HARDWARE PRINCIPLE: CURRENT SINKING VS. SOURCING

Microcontroller I/O pins (internal N-channel MOSFETs) can typically sink more current to Ground (0V) safely than P-channel MOSFETs can source from supply (3.3V). Connecting the LED cathode to pin PA20 ensures maximum brightness and electrical stability.

How Zephyr Handles Active-Low: In bare-metal code, students must write `0` to turn the LED ON and 1 to turn it OFF. In Zephyr, the board's Devicetree defines:

```
led0: led_0 {
    gpios = <&porta 20 GPIO_ACTIVE_LOW>;
    label = "LED0";
};
```

Because `GPIO_ACTIVE_LOW` is declared in Devicetree, Zephyr's `gpio_pin_set_dt(&led, 1)` activates the LED by driving the pin LOW automatically. You program in logical states: **1 is always ON, 0 is always OFF**!

4. Software Architecture & Timing Specifications

4.1 Layered Driver Architecture

Zephyr cleanly isolates application code from physical silicon registers through a multi-tiered driver model:

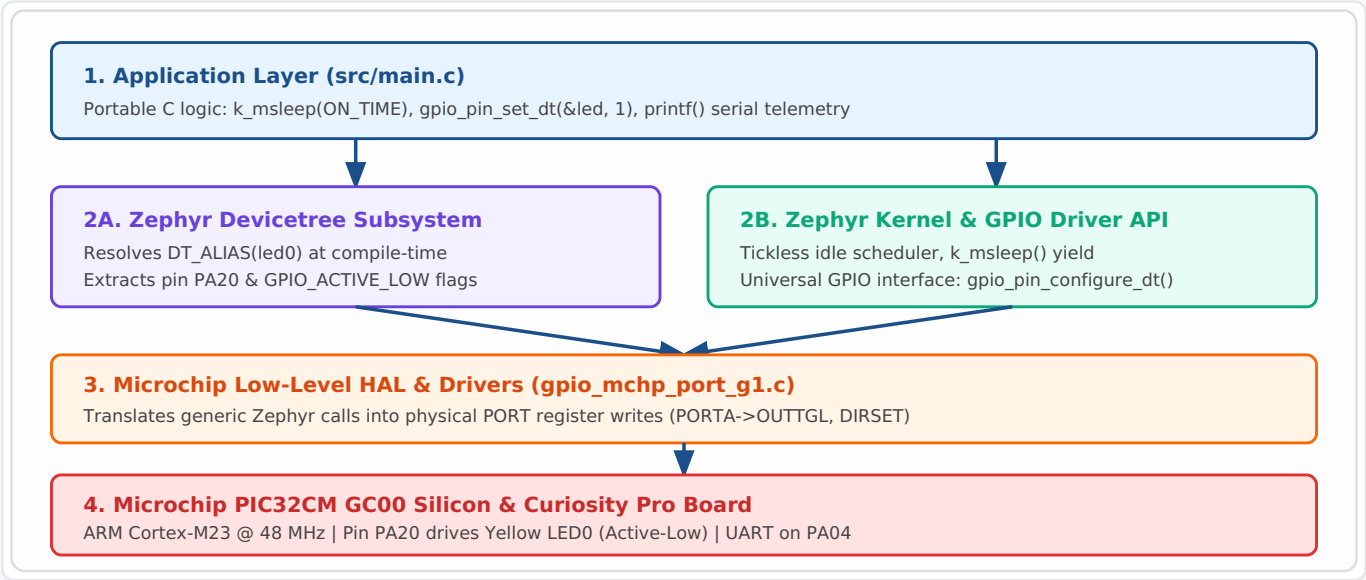


Figure 3: Multi-tier architectural flow from C application to Microchip PORT hardware registers.

4.2 Timing Waveform Specification

The target application generates a deterministic pulsing rhythm: **5 rapid blink cycles** (350 ms ON, 450 ms OFF), followed by an extended **1000 ms quiescent pause**:

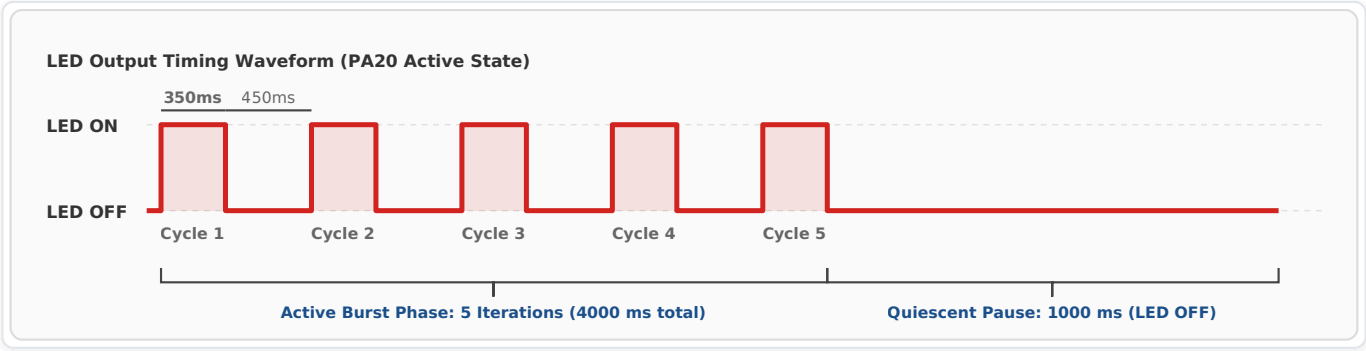


Figure 4: Exact timing diagram showing 5 active pulse cycles (4000 ms) and quiescent pause (1000 ms).

4.3 Program Execution Flowchart

The firmware executes through the deterministic cyclic state machine depicted below:

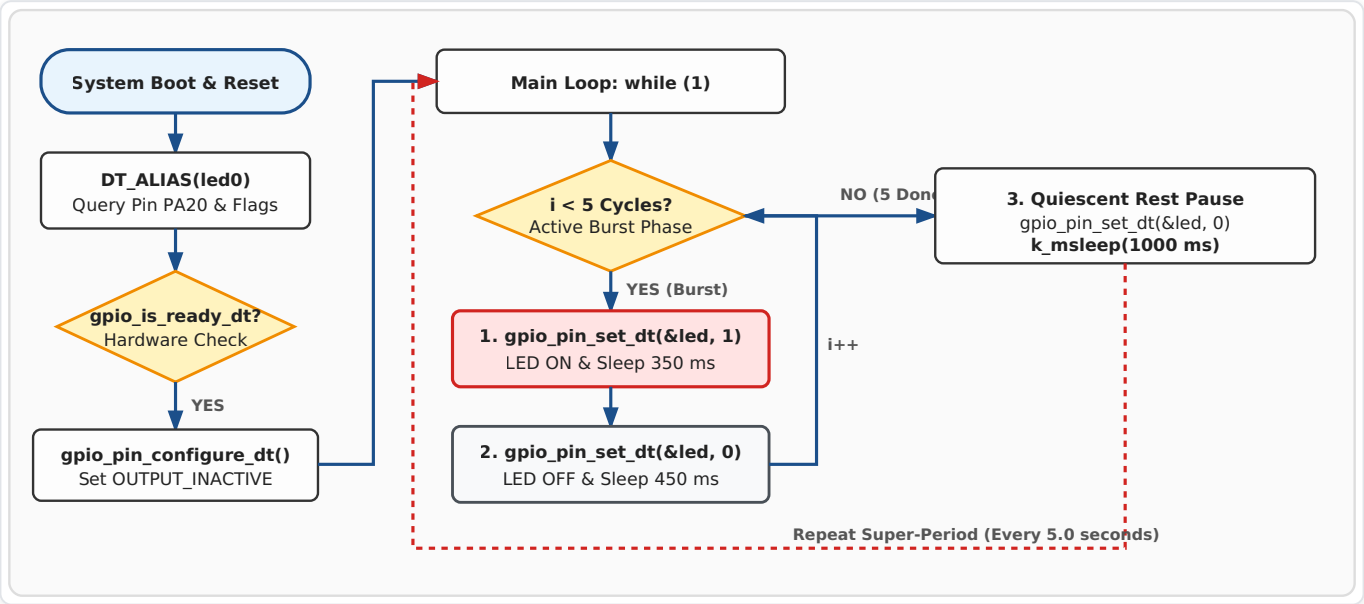


Figure 5: State machine flowchart showing initialization, 5-burst loop, and 1000 ms rest cycle.

5. Step-by-Step Development Environment Setup Guide

Follow these instructions to configure your host machine (Linux, Ubuntu 22.04/24.04, or Windows WSL2):

Step 1: Install Host Operating System Packages

```
sudo apt update && sudo apt install -y git cmake ninja-build gperf ccache dfu-util   device-tree-
compiler wget python3-dev python3-pip python3-setuptools xz-utils make gcc
```

Step 2: Initialize Workspace & Install West Meta-Tool

```
# Create workspace and Python virtual environment
mkdir -p ~/zephyrproject && cd ~/zephyrproject
python3 -m venv .venv
source .venv/bin/activate

# Install West and download Zephyr modules
pip install west
west init -l zephyr
west update
west zephyr-export
pip install -r zephyr/scripts/requirements.txt
```

Step 3: Install Zephyr ARM SDK Cross-Compiler

```
wget https://github.com/zephyrproject-rtos/sdk-ng/releases/download/v0.16.8/zephyr-sdk-0.16.8_linux-
x86_64.tar.xz
tar xvf zephyr-sdk-0.16.8_linux-x86_64.tar.xz
cd zephyr-sdk-0.16.8 && ./setup.sh -t arm-zephyr-eabi -h -c
```

Step 4: Configure USB udev Rules for Debugger Access

```
sudo cp ~/zephyrproject/zephyr/scripts/udev/rules.d/* /etc/udev/rules.d/
sudo udevadm control --reload-rules && sudo udevadm trigger
```

6. Detailed Code Walkthrough (`src/main.c`)

Let's examine how each section of the firmware operates:

Part 1: Headers & Timing Constants

```
#include <stdio.h>
#include <zephyr/kernel.h>
#include <zephyr/drivers/gpio.h>

#define SLEEP_TIME_MS   1000    /* Quiescent Pause: 1000 ms */
#define ON_TIME         350     /* Active Pulse ON: 350 ms */
#define OFF_TIME        450     /* Inter-pulse OFF: 450 ms */
```

Explanation: `<zephyr/kernel.h>` provides kernel sleep services (`k_msleep()`), and `<zephyr/drivers/gpio.h>` provides standard GPIO driver APIs. Note that preprocessor macros must not use `=` assignment operators.

Part 2: Devicetree Node Resolution

```
#define LED0_NODE DT_ALIAS(led0)
static const struct gpio_dt_spec led = GPIO_DT_SPEC_GET(LED0_NODE, gpios);
```

Explanation: `DT_ALIAS(led0)` queries the board's Devicetree for the alias `led0`. `GPIO_DT_SPEC_GET` packages the PORTA driver pointer, pin index 20, and `GPIO_ACTIVE_LOW` polarity into a static constant structure at compile time.

Part 3: Device Initialization & Readiness

```
int main(void)
{
    int ret;
    bool led_state;

    if (!gpio_is_ready_dt(&led)) {
        return 0;
    }

    ret = gpio_pin_configure_dt(&led, GPIO_OUTPUT_INACTIVE);
    if (ret < 0) {
        return 0;
    }
```

Explanation: `gpio_is_ready_dt()` verifies that the physical PORT controller initialized without hardware faults. `gpio_pin_configure_dt(&led, GPIO_OUTPUT_INACTIVE)` sets the pin as an output in the OFF state.

Part 4: The Nested 5-Burst Loop & Rest Pause

```
    while (1) {
        /* Loop 5 times: 350 ms ON, 450 ms OFF */
        for (int i = 0; i < 5; i++) {
            /* 1. Turn LED ON for 350 ms */
            ret = gpio_pin_set_dt(&led, 1);
            led_state = true;
            printf("Iteration %d/5 - LED state: %s (350 ms ON)\n", i + 1, "ON");
            k_msleep(ON_TIME);

            /* 2. Turn LED OFF for 450 ms */
            ret = gpio_pin_set_dt(&led, 0);
            led_state = false;
            printf("Iteration %d/5 - LED state: %s (450 ms OFF)\n", i + 1, "OFF");
            k_msleep(OFF_TIME);
        }

        /* 3. Quiescent Pause: LED remains OFF for 1000 ms */
        ret = gpio_pin_set_dt(&led, 0);
        printf("Pause - LED state: OFF (1000 ms Pause)\n\n");
        k_msleep(SLEEP_TIME_MS);
    }
    return 0;
}
```

Explanation: `gpio_pin_set_dt(&led, 1)` turns the LED ON. `k_msleep(ON_TIME)` yields the CPU for 350 ms. After 5 cycles, the LED is turned OFF and sleeps for 1000 ms before repeating.

7. Building, Flashing, and Serial Telemetry

Action	Command Line	Description
Compile Firmware	`west build -b pic32cm_gc00_cpro pic32cm_gc00_blinky -p always`	Performs pristine clean build, links Zephyr kernel & outputs ELF/HEX
Flash Hardware	`west flash`	Programs firmware directly over USB via the onboard nEDBG debugger
Monitor Serial Console	`picocom -b 115200 /dev/ttyACM0`	Connects to the virtual COM port at 115200 baud, 8N1

8. Hands-On Student Lab Challenges

Lab 1: Morse Code SOS

Modify macros to generate international distress signal SOS (3 short pulses of 150ms, 3 long pulses of 450ms, 3 short pulses of 150ms).

Lab 2: Dual Alternating Beacon

Add secondary LED1 (PA18) via `DT_ALIAS(led1)`. Alternate LED0 and LED1 like an emergency responder beacon.

Lab 3: Pushbutton Gate

Read switch SW0 (PB07) via `DT_ALIAS(sw0)`. Only run the 5-burst blinking pattern when the button is held pressed.

© 2026 Microchip Technology Inc. & The Zephyr Project. Developed for Embedded Systems University Curriculum.